

Guide for building the main AWS Architecture

Pasquale Luca Tommasino

January 8, 2026

1 Coding

1.1 S3

1.1.1 amzn-s3-cc-webapp

For this project, we chose to deploy the web app using Amazon S3. The main motivation was cost reduction: S3 allows hosting a static website at significantly lower costs compared to an EC2-based solution. We are aware that exposing objects publicly in the bucket does not represent the most secure configuration possible, but the goal of the project is to understand the architecture and operational workflow behind the deployment of a web application.

Creation of the S3 bucket The initial configuration of the bucket was carried out by following these steps:

- Click on **Create bucket**.
- Set:
 - **AWS Region**: us-east-1 (default).
 - **Bucket type**: General purpose.
 - **Bucket name**: amzn-s3-cc-webapp.
 - **Object ownership**: ACLs disabled.
 - **Block public access**: Block all public access.
- Leave all other settings at their default values.
- Click on **Create bucket**.

Upload HTML file Next, upload to the bucket the HTML file that contains the website.

Enabling static website hosting After creating the bucket and uploading the website, static website hosting was enabled through the following steps:

- Open the bucket and access the **Properties** section.
- Scroll down to the **Static website hosting** section.
- Click on **Edit**.
- Enable the **Host a static website** option.
- Set the **Index document** to the previously uploaded HTML file.
- Save the changes.

Access settings and ACL To make the HTML file publicly accessible, it was necessary to modify the access settings:

- Access the **Permissions** section of the bucket.
- In the **Block public access** section, click on **Edit** and disable the **Block all public access** option.
- While remaining in the same section, locate **Object ownership** and set **ACLs enabled**, accepting the required conditions.

Finally, to make the file effectively reachable:

- Return to the list of objects in the bucket.
- Select the HTML file.
- Open the **Actions** menu.
- Click on **Make public using ACL**.
- Save the configuration.

At this point, the static website is accessible via the endpoint provided by Amazon S3.

1.1.2 amzn-s3-cc-source

Create the bucket named **amzn-s3-cc-source**. This is done simply by following all the default settings, choosing only the region corresponding to the one in which we are working, i.e., the closest to us.

Then, by clicking on the created bucket, go to the **Permissions** section, scroll to the **Cross-origin resource sharing (CORS)** section, and paste the following script:

```
[
  {
    "AllowedHeaders": [
      "*"
    ],
    "AllowedMethods": [
      "GET",
      "POST"
    ],
    "AllowedOrigins": [
      "*"
    ],
    "ExposeHeaders": [
      "ETag"
    ],
    "MaxAgeSeconds": 3000
  }
]
```

This will make it possible to provide information and therefore receive files to be uploaded later via the API.

1.1.3 amzn-s3-cc-destination

Create the bucket named **amzn-s3-cc-destination**. This is done simply by following all the default settings, choosing only the region corresponding to the one in which we are working, i.e., the closest to us.

Then, by clicking on the created bucket, go to the **Permissions** section, scroll to the **Cross-origin resource sharing (CORS)** section, and paste the following script:

```
[
    {
        "AllowedHeaders": [
            "*"
        ],
        "AllowedMethods": [
            "GET"
        ],
        "AllowedOrigins": [
            "*"
        ],
        "ExposeHeaders": []
    }
]
```

This will make it possible to receive files to be downloaded later via the API.

1.2 Lambda

1.2.1 Lambda4API

To implement the server-side application logic, AWS Lambda was used. The function was created by following the standard procedure provided by the AWS Management Console:

- Access the **AWS Lambda** service.
- Click on **Create function**.
- Select the **Author from scratch** option.
- In the basic information section, set:
 - **Function name:** Lambda4API
 - **Runtime:** Python 3.14
 - **Architecture:** leave the default setting.
- In the **Permissions** section, choose the option **Use an existing role**, then select the role **LabRole**, the only one available for AWS Academy accounts and suitable for our use case.

After confirming the settings, it is possible to proceed with the creation of the function and the insertion of the required Python code.

```
import json
import boto3
import uuid

s3 = boto3.client("s3")

WEBAPP_PREFIX = "webapp/"          # images used by the webapp (trigger ON)
IMG_PREFIX    = "img/"             # test images stored here (trigger OFF)
BW_PREFIX     = "bw-"              # prefix for BW files generated by the Lambda converter

def cors_headers():
    return {
        "Content-Type": "application/json",
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Headers": "Authorization, Content-Type",
        "Access-Control-Allow-Methods": "GET, OPTIONS",
    }
```

```

def lambda_handler(event, context):
    bucket_name = event["pathParameters"]["bucket"]
    file_name = event["queryStringParameters"]["file"]
    mode = event["queryStringParameters"]["mode"]

    # Validate extension
    if "." not in file_name:
        return {
            "statusCode": 400,
            "headers": cors_headers(),
            "body": json.dumps(
                {"error": "File name must have an extension (.jpg, .jpeg, .png)"}
            ),
        }

    ext = file_name.rsplit(".", 1)[-1].lower()

    if ext in ("jpg", "jpeg"):
        content_type = "image/jpeg"
    elif ext == "png":
        content_type = "image/png"
    else:
        return {
            "statusCode": 400,
            "headers": cors_headers(),
            "body": json.dumps({"error": "Only JPG, JPEG and PNG are allowed."}),
        }

    if mode == "upload":

        object_key = f"{WEBAPP_PREFIX}-{file_name}" # e.g., webapp/dog.jpg

        URL = s3.generate_presigned_post(Bucket=bucket_name, Key=object_key, Fields=None, Conditions=None)

        body = {"URL": URL}
    elif mode == "download":
        object_key = f"{WEBAPP_PREFIX}{BW_PREFIX}-{file_name}" # e.g., webapp/bw-dog.jpg

        URL = s3.generate_presigned_url("get_object", Params={"Bucket": bucket_name, "Key": object_key})

        body = {"URL": URL}
    elif mode == "copy":
        source_key = f"{IMG_PREFIX}-{file_name}" # it becomes "img/large/earth.png"

        # generate UUID for the new file name
        unique_id = uuid.uuid4().hex
        new_filename = f"{unique_id}.{ext}"

        # destination under webapp/
        dest_key = f"{WEBAPP_PREFIX}-{new_filename}"

        copy_source = {
            "Bucket": bucket_name,
            "Key": source_key,
        }

        s3.copy_object(
            Bucket=bucket_name,

```

```

        Key=dest_key,
        CopySource=copy_source,
    )

    # this is what you really need afterward: new_filename
    body = {
        "new_file": new_filename,
        "s3_path": f"s3://{bucket_name}/{dest_key}",
    }

    else:
        return {
            "statusCode": 400,
            "headers": cors_headers(),
            "body": json.dumps({"error": "Invalid mode"}),
        }

    return {
        "statusCode": 200,
        "headers": cors_headers(),
        "body": json.dumps(body),
    }

```

1.2.2 ConverterBW

To implement the core of the entire project, namely the function that transforms images from RGB to BW, AWS Lambda was used. The function was created by following the standard procedure provided by the AWS Management Console:

- Access the **AWS Lambda** service.
- Click on **Create function**.
- Select the **Author from scratch** option.
- In the basic information section, set:
 - **Function name:** ConverterBW
 - **Runtime:** Python 3.14
 - **Architecture:** leave the default setting.
- In the **Permissions** section, choose the option **Use an existing role**, then select the role **LabRole**, the only one available for AWS Academy accounts and suitable for our use case.

After confirming the settings, create locally a Python file containing the code shown below.

```

import boto3
import uuid
from urllib.parse import unquote_plus
from PIL import Image

s3_client = boto3.client('s3')

# Destination bucket
DEST_BUCKET = 'amzn-s3-cc-destination'

def convert_to_bw(image_path, output_path):
    with Image.open(image_path) as image:
        # "L" = grayscale (8-bit pixels, black and white)
        bw_image = image.convert('L')

```

```

        bw_image.save(output_path)

def lambda_handler(event, context):
    for record in event['Records']:
        source_bucket = record['s3']['bucket']['name']
        key = unquote_plus(record['s3']['object']['key'])

        tmpkey = key.replace('/', '')
        download_path = '/tmp/{}/{}/'.format(uuid.uuid4(), tmpkey)
        upload_path = '/tmp/bw-{}'.format(tmpkey)

        # Download file from source bucket
        s3_client.download_file(source_bucket, key, download_path)

        # Convert image to black and white
        convert_to_bw(download_path, upload_path)

        # Trigger only on webapp/ folder
        if '/' in key:
            folder, filename = key.rsplit('/', 1)    # e.g., "webapp", "sample.jpg"
            dest_key = f"{folder}/bw-{filename}"      # e.g., "webapp/bw-sample.jpg"
        else:
            filename = key
            dest_key = f"bw-{filename}"
        # Upload on destination bucket, keeping the same file name, adding a 'bw' prefix
        s3_client.upload_file(upload_path, DEST_BUCKET, f'bw-{key}')

```

In the directory containing the Python file, create the `package` directory as follows:

```
mkdir package
```

Then type the following command in order to download the entire Pillow library, which is required for Lambda:

```

pip install \
--platform manylinux2014_x86_64 \
--target=package \
--implementation cp \
--python-version 3.14 \
--only-binary=:all: --upgrade \
pillow

```

Create a `.zip` file containing the application code and the Pillow library. On Linux or macOS, run the following commands from the command-line interface:

```

cd package
zip -r ../lambda_function.zip .
cd ..
zip lambda_function.zip lambda_function.py

```

S3 Trigger for the Lambda function To automate the processing of images uploaded to the S3 bucket, a trigger was configured that allows the Lambda function to be automatically invoked when new objects are created in the source bucket. This configuration makes it possible to integrate the upload workflow with the generation of processed versions of the files.

The procedure to configure the trigger was as follows:

- Open the page of the **Lambda4API** Lambda function and select the **Add trigger** section.
- Choose the **S3** service as the event source.
- In the **Bucket** field, select the source bucket, for example:
 - `s3/amzn-s3-cc-source`

- In the **Event types** section, select:
 - **All object create events**, so that the Lambda function is triggered every time a new file is uploaded to the bucket.
- Leave the **Prefix** and **Suffix** fields empty, since in this configuration the function must respond to any created object without specific filters.
- In the **Recursive invocation** section, select the option acknowledging awareness of the risk of recursive invocations:
 - **I acknowledge that using the same S3 bucket for both input and output is not recommended.**

This warning indicates that if the Lambda function were to write to the same bucket from which it reads, automatic invocation loops could occur. In our case, the processing is controlled and does not generate unwanted recursion.

- Once the configuration has been verified, click on **Add** to add the trigger.

After adding the trigger, AWS automatically updates the necessary policies so that the S3 bucket can invoke the Lambda function. This completes the integration between object uploads and the automatic processing performed by the function.

1.3 Cognito

1.3.1 WebAppCC

For managing user authentication in the web app, Amazon Cognito was used through the creation of a *User Pool*. Cognito allows delegating credential management, the login process, and account recovery to AWS, significantly simplifying the implementation of application-side security.

The configuration of the User Pool was carried out by following the steps outlined below.

- Access the **Amazon Cognito** service from the AWS Management Console.
- Select the **Create user pool** option.
- In the **Sign-in experience** section, choose the following sign-in method:
 - **Email** as the only identifier for user authentication.
- Do not enable any form of **Multi-Factor Authentication (MFA)**, keeping the configuration simple and suitable for the project context.
- Enable **User account recovery** using exclusively:
 - **Email only**, allowing account recovery via email.
- In the **Required attributes** section, select the mandatory attributes for registration:
 - **Email**
 - **Name**
 - **Family Name**
 - **Date of birth**
- Leave all other settings at the default values proposed by AWS.

Once the User Pool configuration was completed, an *App Client* was created, which is required for the web app to interact with Cognito.

- Create a new **App client** associated with the User Pool.
- Assign the following name to the app client:
 - **WebAppCC**

The App Client represents the integration point between the web app and Amazon Cognito and is used to manage the authentication flow and the generation of JWT tokens, which are subsequently validated by API Gateway through the Cognito authorizer.

1.4 API Gateway

1.4.1 API4WebPage

To allow our web page to invoke the Lambda function, a REST API was created using AWS API Gateway. The configuration was carried out by following the steps described below.

- Access the **API Gateway** service.
- Select the **Create REST API** option.
- Choose the **New API** option.
- Set the main parameters:
 - **API name:** API4WebPage
 - **Endpoint type:** Regional
 - **Security policy:** SecurityPolicy_TLS13_1_2_2021_06
 - **Endpoint access mode:** Basic
 - **IP address type:** IPv4
- Leave all other settings at their default values.
- Confirm the creation of the API by clicking on **Create API**.

This configuration allows exposing a public endpoint compatible with the previously created Lambda function, while maintaining an adequate level of security thanks to the use of the most up-to-date TLS policies.

Create a resource named `{bucket}` under the `/` folder.

- By selecting the `{bucket}` resource, choose **Create Method**.
- Set the **Method type** to GET.
- In the **Integration type** section, choose **Lambda function**.
- Enable the **Lambda proxy integration** option. This mode allows the Lambda function to receive the entire structured event, including path and query parameters, and to directly return a response in the `statusCode`, `headers`, and `body` format, as expected by our code.
- Set the **Response transfer mode** to **Buffered**, which requires waiting for the complete response from the Lambda function before transmitting it to the client. This setting is appropriate since the function generates small responses (JSON) without the need for streaming.
- Select the `us-east-1` region and choose the `Lambda4API` Lambda function.
- Enable the **Grant API Gateway permission to invoke your Lambda function** option to allow the API to invoke the function.
- Leave the default value of the **integration timeout** unchanged (29000 ms).

This configuration ensures that the API correctly forwards requests to the Lambda function and that the response is reliably returned to the client, maintaining a structure consistent with the JSON format expected by the web app.

Cognito Authorizer To protect access to our API and allow only users authenticated through Amazon Cognito to invoke the endpoints, a *Cognito Authorizer* was configured within API Gateway. The authorizer automatically validates the JWT tokens generated by the web app login process.

The configuration was carried out through the following steps:

- Access the **Authorizers** section of the created API.
- Click on **Create authorizer**.
- Set the following parameters:
 - **Authorizer name**: Auth4Cognito
 - **Authorizer type**: select **Cognito**
 - **Cognito user pool**: select the **us-east-1** region and choose the previously created User Pool.
 - **Token source**: **Authorization**, i.e., the HTTP header containing the JWT token sent by the client.
- Leave the **Token validation** field empty (optional), since no custom validation of the **aud** attribute is required.
- Confirm the creation by selecting **Create authorizer**.

The Cognito authorizer configured in this way allows associating access control with a specific API method, ensuring that only requests containing a valid JWT token can be processed. This mechanism integrates the identity management implemented through Cognito with API Gateway-level security.

Association of the Authorizer with the GET method After creating the Auth4Cognito authorizer, it must be associated with the previously defined **GET** method, so that every request to the endpoint is authorized through the JWT token generated by Cognito. This operation is performed in the *Method Request* settings.

- Access the **GET** method of the API and select the **Method request** tab.
- In the **Method request settings** section, click on **Edit**.
- In the **Authorization** field, select the Auth4Cognito authorizer created earlier.
- Leave the **Authorization scopes** field empty, since no additional OAuth scopes are required for this application.
- Keep the **Request validator** set to **None**, as no preliminary validation is required at the API Gateway level.
- Ensure that the **API key required** option remains disabled, since authentication is already handled through Cognito.
- The **Operation name** field is optional; it can be left unchanged or renamed if useful for internal documentation.

Once the changes are saved, the **GET** method is protected by the Cognito authorizer: every invocation of the endpoint requires the presence of an **Authorization** header containing a valid JWT token generated by the web app login process.

Enabling the OPTIONS method via CORS To allow the web app hosted on S3 to make requests to the API, it is necessary to enable CORS (Cross-Origin Resource Sharing) support. This operation allows the creation of the `OPTIONS` method, required by browsers for *preflight* requests, and automatically configures the headers needed for interaction with API Gateway.

After selecting the `{bucket}` resource and clicking on **Enable CORS**, a configuration screen is displayed containing the following options:

- **Gateway responses:** select the options

- Default 4XX
- Default 5XX

This allows API Gateway to include CORS headers also in error responses.

- **Access-Control-Allow-Methods:** the supported methods are shown.

- `GET` is selected automatically.

- **Access-Control-Allow-Headers:** field that defines the allowed headers. API Gateway automatically inserts:

- `Content-Type`, `X-Amz-Date`, `Authorization`, `X-API-Key`, `X-Amz-Security-Token`

- **Access-Control-Allow-Origin:** specifies the domains that can access the resource.

- The value `*` allows access from any origin, suitable for an educational or testing environment.

After confirming the configuration, API Gateway automatically generates:

- the `OPTIONS` method for the selected resource,
- the CORS headers in the responses of the `GET` method,
- the rules necessary to ensure that the web app can make requests to the API without being blocked by the browser.

Deploy API After configuring all methods, the Cognito authorizer, and CORS support, it is necessary to deploy the API so that all changes become operational and accessible from the web app. The deployment is performed using the **Deploy API** function available in API Gateway.

- Access the main API panel and select **Deploy API**.
- In the window that appears, select the ***New stage*** option to create a new deployment stage.
- Enter a name for the stage, for example `v2`. The stage represents a version of the API and allows managing multiple releases in parallel.
- Fill in the **Deployment description** field with a brief description, for example: **Deploy of all API**.
- Confirm the operation by clicking on **Deploy**.

Once the deployment is complete, API Gateway automatically generates the stage *Invoke URL*, which represents the public endpoint that can be used by the web app to invoke the `GET` method. This URL will then be inserted into the JavaScript code of the site hosted on S3 for communication with the Lambda.